

Competitive Programming

Section 42 Editorial: Advanced Data Structures Mixed Contest

Competitive Programming Course

2026-09-30

Table of contents

1	Scope and prerequisites	5
2	A. Dynamic Order Statistics	6
2.1	Contract and reduction	6
2.2	Algorithm	6
2.3	Sample walkthrough	6
2.4	Correctness	6
2.5	Complexity	6
2.6	Implementation notes and targeted tests	7
2.7	C++ reference solution	7
2.8	Python reference solution	7
3	B. Range Assign and Add	9
3.1	Contract and reduction	9
3.2	Algorithm	9
3.3	Sample walkthrough	9
3.4	Correctness	9
3.5	Complexity	9
3.6	Implementation notes and targeted tests	10
3.7	C++ reference solution	10
3.8	Python reference solution	11
4	C. Static Rectangle Count	14
4.1	Contract and reduction	14
4.2	Algorithm	14
4.3	Sample walkthrough	14
4.4	Correctness	14
4.5	Complexity	14
4.6	Implementation notes and targeted tests	15
4.7	C++ reference solution	15
4.8	Python reference solution	15
5	D. Nested Ranges Count	16
5.1	Contract and reduction	16
5.2	Algorithm	16
5.3	Sample walkthrough	16
5.4	Correctness	16
5.5	Complexity	17
5.6	Implementation notes and targeted tests	17
5.7	C++ reference solution	17
5.8	Python reference solution	19
6	E. Subtree Add, Point Query	21
6.1	Contract and reduction	21
6.2	Algorithm	21

6.3	Sample walkthrough	21
6.4	Correctness	21
6.5	Complexity	21
6.6	Implementation notes and targeted tests	21
6.7	C++ reference solution	22
6.8	Python reference solution	23
7	F. Dynamic Subtree Maximum	24
7.1	Contract and reduction	24
7.2	Algorithm	24
7.3	Sample walkthrough	24
7.4	Correctness	24
7.5	Complexity	24
7.6	Implementation notes and targeted tests	25
7.7	C++ reference solution	25
7.8	Python reference solution	25
8	G. Subtree Value Count	27
8.1	Contract and reduction	27
8.2	Algorithm	27
8.3	Sample walkthrough	27
8.4	Correctness	27
8.5	Complexity	27
8.6	Implementation notes and targeted tests	28
8.7	C++ reference solution	28
8.8	Python reference solution	28
9	H. Incremental Connectivity	30
9.1	Contract and reduction	30
9.2	Algorithm	30
9.3	Sample walkthrough	30
9.4	Correctness	30
9.5	Complexity	30
9.6	Implementation notes and targeted tests	30
9.7	C++ reference solution	31
9.8	Python reference solution	31
10	I. Subtree Assign, Add, and Sum	32
10.1	Contract and reduction	32
10.2	Algorithm	32
10.3	Sample walkthrough	32
10.4	Correctness	32
10.5	Complexity	32
10.6	Implementation notes and targeted tests	33
10.7	C++ reference solution	33
10.8	Python reference solution	35
11	J. Static Vertex Path Sums	37
11.1	Contract and reduction	37
11.2	Algorithm	37
11.3	Sample walkthrough	37
11.4	Correctness	37

11.5 Complexity	37
11.6 Implementation notes and targeted tests	38
11.7 C++ reference solution	38
11.8 Python reference solution	39
12 K. Static Weighted Rectangles	40
12.1 Contract and reduction	40
12.2 Algorithm	40
12.3 Sample walkthrough	40
12.4 Correctness	40
12.5 Complexity	40
12.6 Implementation notes and targeted tests	40
12.7 C++ reference solution	41
12.8 Python reference solution	41
13 L. Subtree Add, Subtree Maximum	42
13.1 Contract and reduction	42
13.2 Algorithm	42
13.3 Sample walkthrough	42
13.4 Correctness	42
13.5 Complexity	42
13.6 Implementation notes and targeted tests	43
13.7 C++ reference solution	43
13.8 Python reference solution	44
14 Stress testing and what to carry forward	46

1 Scope and prerequisites

This contest uses only techniques taught through section 41. See the lesson's prerequisite table for their first teaching locations. The solutions combine ordinary Fenwick trees, segment trees, lazy tags, offline sweeps, Euler intervals, LCA, and insertion-only DSU. HLD, persistence, rollback, merge-sort trees, and multidimensional Fenwick trees are not required.

A uses count-guided segment-tree descent from section 40. G uses an offline value sweep. Every subtree update maps to one Euler interval; J has static vertex values and uses root sums with LCA. Keep these contracts explicit when assessing whether an earlier structure is sufficient.

2 A. Dynamic Order Statistics

2.1 Contract and reduction

Maintain frequencies on the score axis. Repeated scores occupy repeated ranks, so a set cannot represent the answers. Section 40 already taught descending a count segment tree to select the k -th active item; here a leaf can hold more than one item. No Fenwick binary-lifting search is needed.

2.2 Algorithm

```
build an all-zero count tree over scores 1..m
insert x: increment leaf x and repair ancestor counts
erase x: decrement only if leaf x is positive
count <= x: sum the score interval [1,x]
k-th: reject if k exceeds the root count
      descend left if left.count >= k
      otherwise subtract left.count and descend right
```

2.3 Sample walkthrough

The sample builds {3,3,5}. Rank two belongs to score 3. Erasing one copy leaves {3,5}, whose rank two is 5. Counting scores at most 4 before that removal gives 2.

2.4 Correctness

Each leaf is the multiplicity of its score; each internal node is the sum of its children's counts. Insertions and valid removals preserve this by repairing the leaf-to-root path. At a selection node, the first `left.count` items belong to its left interval. Staying left or subtracting that count therefore preserves the desired rank. At a leaf, its score is the answer.

2.5 Complexity

Build $O(m)$; each operation $O(\log m)$; memory $O(m)$.

2.6 Implementation notes and targeted tests

Do not remove an absent copy, and do not deduplicate repeated scores. Reject ranks above the total before descending. Test an empty multiset, multiple copies at one score, scores 1 and m, and invalid ranks.

2.7 C++ reference solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int m,q;if(!(cin>>m>>q))return 0;
int size=1;while(size<m)size*=2;vector<int> count(2*size);
auto change=[&](int x,int delta){int z=size+x-1;count[z]+=delta;for(z/=2;z/=2)count[z]=count[z]+delta;
auto prefix=[&](int x){int l=size,r=size+x,total=0;while(l<r){if(l&1)total+=count[l++];if(r&1)total+=count[r--];l/=2;r/=2;}
while(q--){int op,x;cin>>op>>x;
if(op==1)change(x,1);else if(op==2){if(count[size+x-1])change(x,-1);}
else if(op==4)cout<<prefix(x)<<'\n';else if(x>count[1])cout<<-1<<'\n';
else{int z=1;while(z<size){if(count[2*z]>=x)z*=2;else{x-=count[2*z];z=2*z+1;}}cout<<z-size+1<<'\n';}
}
```

2.8 Python reference solution

```
import sys

def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data:return
    m,q=data[:2];size=1
    while size<m:size*=2
    count=[0]*(2*size);out=[];pos=2
    def change(x,delta):
        node=size+x-1;count[node]+=delta;node//=2
        while node:count[node]=count[2*node]+count[2*node+1];node//=2
    def prefix(x):
        left,right=size,size+x;total=0
        while left<right:
            if left&1:total+=count[left];left+=1
            if right&1:right-=1;total+=count[right]
            left//=2;right//=2
        return total
    for _ in range(q):
        op,x=data[pos:pos+2];pos+=2
        if op==1:change(x,1)
        elif op==2:
            if count[size+x-1]:change(x,-1)
        elif op==4:out.append(prefix(x))
```

```

elif x>count[1]:out.append(-1)
else:
    node=1
    while node<size:
        if count[2*node]>=x:node*=2
        else:x-=count[2*node];node=2*node+1
    out.append(node-size+1)
print('\n'.join(map(str,out)))
if __name__=='__main__':main()

```

3 B. Range Assign and Add

3.1 Contract and reduction

The operation contract combines the lazy propagation of section 37 with two update types. Addition and assignment do not commute. Keep a node sum, an optional assignment, and an addition when no assignment is pending. A boolean presence flag distinguishes assignment to zero from no assignment.

3.2 Algorithm

```
assign(node,x): sum = x*length; assignment = x; has_assignment = true; add = 0
add(node,x): sum += x*length
              if has_assignment: assignment += x
              else: add += x
partial operation: push pending tags to children, recurse, pull sums
full-cover query: return node.sum
```

3.3 Sample walkthrough

The sample starts with sum 15. Assigning positions 2..4 to 7 gives 1,7,7,7,5, sum 27. Adding -2 to positions 1..3 gives -1,5,5,7,5, sum 21; positions 2..3 sum to 10.

3.4 Correctness

The node sum describes its current interval, including its pending update. An assignment replaces every prior value and pending update. An addition shifts each value, so the sum increases by $x \times \text{length}$; folding it into a pending assignment describes the same transformation. Pushing applies that transformation to both children, and pulling sums their disjoint intervals. Induction on tree recursion proves every range update and sum query.

3.5 Complexity

Build $O(n)$; every operation $O(\log n)$; memory $O(n)$.

3.6 Implementation notes and targeted tests

Applying assignment without clearing earlier additions changes update order. Do not treat zero as an absent assignment. Test add-then-assign, assign-then-add, overlapping intervals, singleton ranges, and negative values. Use 64-bit sums, including the multiplication by segment length.

3.7 C++ reference solution

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int n, q;
vector<ll> sm, addv, asgv;
vector<char> hasa;

static void put_assign(int x, int len, ll v) {
    sm[x] = v * len; asgv[x] = v; hasa[x] = 1; addv[x] = 0;
}
static void put_add(int x, int len, ll v) {
    sm[x] += v * len;
    if (hasa[x]) asgv[x] += v; else addv[x] += v;
}
static void push(int x, int l, int r) {
    int m = (l + r) / 2, lc = 2 * x, rc = 2 * x + 1;
    if (hasa[x]) {
        put_assign(lc, m - l + 1, asgv[x]);
        put_assign(rc, r - m, asgv[x]);
        hasa[x] = 0; asgv[x] = 0;
    }
    if (addv[x]) {
        put_add(lc, m - l + 1, addv[x]);
        put_add(rc, r - m, addv[x]);
        addv[x] = 0;
    }
}
static void build(int x, int l, int r, vector<ll> &a) {
    if (l == r) { sm[x] = a[l]; return; }
    int m = (l + r) / 2;
    build(2 * x, l, m, a); build(2 * x + 1, m + 1, r, a);
    sm[x] = sm[2 * x] + sm[2 * x + 1];
}
static void upd(int x, int l, int r, int ql, int qr,
               ll v, bool assign_) {
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) {
        if (assign_) put_assign(x, r - l + 1, v);
        else put_add(x, r - l + 1, v);
        return;
    }
}
```

```

    }
    push(x, l, r);
    int m = (l + r) / 2;
    upd(2 * x, l, m, ql, qr, v, assign_);
    upd(2 * x + 1, m + 1, r, ql, qr, v, assign_);
    sm[x] = sm[2 * x] + sm[2 * x + 1];
}

static ll qry(int x, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return sm[x];
    push(x, l, r);
    int m = (l + r) / 2;
    return qry(2 * x, l, m, ql, qr) + qry(2 * x + 1, m + 1, r, ql, qr);
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    if (!(cin >> n >> q)) return 0;
    vector<ll> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    sm.assign(4 * n + 4, 0); addv.assign(4 * n + 4, 0);
    asgv.assign(4 * n + 4, 0); hasa.assign(4 * n + 4, 0);
    build(1, 1, n, a);
    string out;
    while (q--) {
        int t, l, r; ll x;
        cin >> t;
        if (t == 3) {
            cin >> l >> r;
            out += to_string(qry(1, 1, n, l, r));
            out += '\n';
        } else {
            cin >> l >> r >> x;
            upd(1, 1, n, l, r, x, t == 1);
        }
    }
    cout << out;
}

```

3.8 Python reference solution

```

import sys

def main():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    pos = 0

```

```

n = int(data[pos]); q = int(data[pos + 1]); pos += 2
a = [0] * (n + 1)
for i in range(1, n + 1):
    a[i] = int(data[pos]); pos += 1

size = 4 * n + 4
sm = [0] * size
addv = [0] * size
asgv = [0] * size
hasa = [False] * size

def put_assign(x, ln, v):
    sm[x] = v * ln; asgv[x] = v; hasa[x] = True; addv[x] = 0

def put_add(x, ln, v):
    sm[x] += v * ln
    if hasa[x]:
        asgv[x] += v
    else:
        addv[x] += v

def push(x, l, r):
    m = (l + r) // 2; lc = 2 * x; rc = 2 * x + 1
    if hasa[x]:
        put_assign(lc, m - l + 1, asgv[x])
        put_assign(rc, r - m, asgv[x])
        hasa[x] = False; asgv[x] = 0
    if addv[x]:
        put_add(lc, m - l + 1, addv[x])
        put_add(rc, r - m, addv[x])
        addv[x] = 0

def build(x, l, r):
    if l == r:
        sm[x] = a[l]; return
    m = (l + r) // 2
    build(2 * x, l, m); build(2 * x + 1, m + 1, r)
    sm[x] = sm[2 * x] + sm[2 * x + 1]

def upd(x, l, r, ql, qr, v, is_assign):
    if qr < l or r < ql:
        return
    if ql <= l and r <= qr:
        if is_assign:
            put_assign(x, r - l + 1, v)
        else:
            put_add(x, r - l + 1, v)
        return
    push(x, l, r)
    m = (l + r) // 2
    upd(2 * x, l, m, ql, qr, v, is_assign)

```

```

    upd(2 * x + 1, m + 1, r, ql, qr, v, is_assign)
    sm[x] = sm[2 * x] + sm[2 * x + 1]

def qry(x, l, r, ql, qr):
    if qr < l or r < ql:
        return 0
    if ql <= l and r <= qr:
        return sm[x]
    push(x, l, r)
    m = (l + r) // 2
    return (qry(2 * x, l, m, ql, qr)
            + qry(2 * x + 1, m + 1, r, ql, qr))

sys.setrecursionlimit(300000)
build(1, 1, n)
out = []
for _ in range(q):
    t = int(data[pos]); pos += 1
    if t == 3:
        l = int(data[pos]); r = int(data[pos + 1]); pos += 2
        out.append(qry(1, 1, n, l, r))
    else:
        l = int(data[pos]); r = int(data[pos + 1])
        x = int(data[pos + 2]); pos += 3
        upd(1, 1, n, l, r, x, t == 1)
sys.stdout.write("\n".join(map(str, out)) + ("\n" if out else ""))

if __name__ == "__main__":
    main()

```

4 C. Static Rectangle Count

4.1 Contract and reduction

All sensors and queries are fixed, allowing the offline sweep of section 13. For rectangle $[a, c] \times [b, d]$, subtract the sensors in its y -range with $x \leq a-1$ from those with $x \leq c$. Compress y -coordinates and use a Fenwick tree to count the active sensors in $[b, d]$.

4.2 Algorithm

```
create events (c,b,d,id,+1) and (a-1,b,d,id,-1)
sort sensors by x; sort events by their x bound
for each event X:
    insert every not-yet-inserted sensor with  $x \leq X$  into the  $y$  Fenwick
     $\text{answer}[\text{id}] += \text{sign} * (\text{prefix}(\text{upper\_bound}(d)) - \text{prefix}(\text{lower\_bound}(b)))$ 
print answers by original query id
```

4.3 Sample walkthrough

The first sample rectangle contains (1,1) and (2,3), giving 2. The second contains (4,2) and (5,5), also giving 2. There is no sensor on the third rectangle's row $y=4$, so its answer is 0.

4.4 Correctness

Before an event at X , the active set is exactly the sensors with $x \leq X$. This holds initially and is preserved by inserting all eligible sensors in sorted order. The Fenwick prefix difference includes exactly y -values in $[b, d]$. Subtracting the event at $a-1$ removes exactly sensors left of the rectangle. Each remaining sensor is counted once, including boundary points and coincident sensors.

4.5 Complexity

Sorting and processing cost $O((n+q) \log(n+q))$; memory $O(n+q)$.

4.6 Implementation notes and targeted tests

Insert sensors with equal x before answering an event at that x . Do not require query boundaries to occur in the compressed coordinates. Use separate lower and upper bounds for inclusive y -ranges. Test repeated points, a point on each border, zero-width rectangles, and empty results.

4.7 C++ reference solution

```
#include <bits/stdc++.h>
using namespace std; struct BIT{int n; vector<int>b; BIT(int n):n(n),b(n+1){} void add(int i,int x){while(i<=n){b[i]+=x;i+=i&-i}}
struct E{int x,y1,y2,id,sgn; bool operator<(const E&o) const{return x<o.x;}};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,q;if(!(cin>>n>>q)) return 0;v
```

4.8 Python reference solution

```
import sys,bisect
class BIT:
    def __init__(self,n): self.n=n; self.bit=[0]*(n+1)
    def add(self,i,x):
        while i<=self.n: self.bit[i]+=x; i+=i&-i
    def sum(self,i):
        s=0
        while i>0: s+=self.bit[i]; i-=i&-i
        return s
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,q=data[0],data[1]; idx=2; pts=[]; ys=[]
    for _ in range(n): x,y=data[idx],data[idx+1]; idx+=2; pts.append((x,y)); ys.append(y)
    events=[]; ans=[0]*q
    for qi in range(q):
        x1,y1,x2,y2=data[idx],data[idx+1],data[idx+2],data[idx+3]; idx+=4; ys+= [y1,y2]
        events.append((x2,y1,y2,qi,1)); events.append((x1-1,y1,y2,qi,-1))
    ys=sorted(set(ys)); pts.sort(); events.sort(); bit=BIT(len(ys)); p=0
    for x,y1,y2,qi,sgn in events:
        while p<n and pts[p][0]<=x:
            bit.add(bisect.bisect_left(ys,pts[p][1])+1,1); p+=1
        l=bisect.bisect_left(ys,y1)+1; r=bisect.bisect_right(ys,y2)
        ans[qi]+=sgn*(bit.sum(r)-bit.sum(l-1))
    print("\n".join(map(str,ans)))
if __name__=="__main__": main()
```

5 D. Nested Ranges Count

5.1 Contract and reduction

Sort intervals by increasing left endpoint, breaking ties by decreasing right endpoint. Use a Fenwick tree over compressed right endpoints. In a forward sweep, earlier intervals with right endpoint at least the current endpoint contain it. In a reverse sweep, later intervals with right endpoint at most the current endpoint are contained by it. Identical intervals must be handled as a group, because they count as containing each other.

5.2 Algorithm

```
sort (left ascending, right descending); group identical pairs
forward group of size s and right rank r:
    contained_by = seen - prefix(r-1) + s-1
    then insert all s copies
clear Fenwick
reverse group of size s and right rank r:
    contains = prefix(r) + s-1
    then insert all s copies
restore original interval order
```

5.3 Sample walkthrough

For $[1,6]$, two copies of $[2,5]$, and $[3,4]$, the outer interval contains three others. Each $[2,5]$ contains its duplicate and $[3,4]$, giving two; it is contained by the outer interval and its duplicate, also two. The inner interval is contained by all three others.

5.4 Correctness

In the forward sweep, every inserted interval has left endpoint no larger than the current one. Its right endpoint is at least the current right endpoint exactly when it contains the current interval. The reverse sweep has the opposite left-endpoint condition, and its prefix selects exactly contained intervals. Processing a whole identical group before insertion avoids arbitrary within-group order; each member receives $s-1$ contributions from its other identical members and none from itself.

5.5 Complexity

Sorting and both sweeps cost $O(n \log n)$; memory $O(n)$.

5.6 Implementation notes and targeted tests

Sorting equal-left intervals by increasing right endpoint breaks the sweep condition. Inserting identical copies one at a time gives asymmetric answers. Test all intervals identical, shared left or right endpoints, disjoint intervals, and a deeply nested family.

5.7 C++ reference solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    vector<int> bit;
    explicit Fenwick(int n) : bit(n + 1, 0) {}

    void add(int index, int delta) {
        for (int n = (int)bit.size(); index < n; index += index & -index) {
            bit[index] += delta;
        }
    }

    int sum(int index) const {
        int result = 0;
        for (; index > 0; index -= index & -index) result += bit[index];
        return result;
    }
};

struct Interval {
    long long left, right;
    int index;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<Interval> ranges(n);
    vector<long long> rights;
    for (int i = 0; i < n; i++) {
        cin >> ranges[i].left >> ranges[i].right;
```

```

        ranges[i].index = i;
        rights.push_back(ranges[i].right);
    }
    sort(rights.begin(), rights.end());
    rights.erase(unique(rights.begin(), rights.end()), rights.end());
    sort(ranges.begin(), ranges.end(), [](const Interval& a, const Interval& b) {
        if (a.left != b.left) return a.left < b.left;
        return a.right > b.right;
    });
    auto rank_of = [&](long long right) {
        return int(lower_bound(rights.begin(), rights.end(), right) - rights.begin()) + 1;
    };

    vector<int> contains(n), contained_by(n);
    Fenwick bit((int)rights.size());
    int processed = 0;
    for (int start = 0; start < n; start++) {
        int end = start + 1;
        while (end < n && ranges[end].left == ranges[start].left &&
            ranges[end].right == ranges[start].right) {
            end++;
        }
        int size = end - start;
        int rank = rank_of(ranges[start].right);
        int base = processed - bit.sum(rank - 1);
        for (int i = start; i < end; i++) contained_by[ranges[i].index] = base + size - 1;
        bit.add(rank, size);
        processed += size;
        start = end;
    }

    bit = Fenwick((int)rights.size());
    for (int end = n; end > 0; end--) {
        int start = end - 1;
        while (start > 0 && ranges[start - 1].left == ranges[end - 1].left &&
            ranges[start - 1].right == ranges[end - 1].right) {
            start--;
        }
        int size = end - start;
        int rank = rank_of(ranges[start].right);
        int base = bit.sum(rank);
        for (int i = start; i < end; i++) contains[ranges[i].index] = base + size - 1;
        bit.add(rank, size);
        end = start;
    }

    for (int i = 0; i < n; i++) cout << contains[i] << (i + 1 == n ? '\n' : ' ');
    for (int i = 0; i < n; i++) cout << contained_by[i] << (i + 1 == n ? '\n' : ' ');
    return 0;
}

```

5.8 Python reference solution

```
import bisect
import sys

class Fenwick:
    def __init__(self, size: int) -> None:
        self.bit = [0] * (size + 1)

    def add(self, index: int, delta: int) -> None:
        while index < len(self.bit):
            self.bit[index] += delta
            index += index & -index

    def prefix_sum(self, index: int) -> int:
        total = 0
        while index:
            total += self.bit[index]
            index -= index & -index
        return total

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    ranges = [(data[2 * i + 1], data[2 * i + 2], i) for i in range(n)]
    rights = sorted({right for _, right, _ in ranges})
    ranges.sort(key=lambda interval: (interval[0], -interval[1]))

    def rank_of(right: int) -> int:
        return bisect.bisect_left(rights, right) + 1

    contains = [0] * n
    contained_by = [0] * n

    bit = Fenwick(len(rights))
    processed = 0
    start = 0
    while start < n:
        end = start + 1
        while end < n and ranges[end][:2] == ranges[start][:2]:
            end += 1
        group_size = end - start
        rank = rank_of(ranges[start][1])
        base = processed - bit.prefix_sum(rank - 1)
        for _, _, index in ranges[start:end]:
            contained_by[index] = base + group_size - 1
        processed += group_size
        start = end
```

```

        bit.add(rank, group_size)
        processed += group_size
        start = end

bit = Fenwick(len(rights))
end = n
while end:
    start = end - 1
    while start > 0 and ranges[start - 1][:2] == ranges[end - 1][:2]:
        start -= 1
    group_size = end - start
    rank = rank_of(ranges[start][1])
    base = bit.prefix_sum(rank)
    for _, _, index in ranges[start:end]:
        contains[index] = base + group_size - 1
    bit.add(rank, group_size)
    end = start

print(" ".join(map(str, contains)))
print(" ".join(map(str, contained_by)))

if __name__ == "__main__":
    main()

```

6 E. Subtree Add, Point Query

6.1 Contract and reduction

Section 41 maps subtree u to one Euler interval. Adding to that interval and querying a single position is the difference-array/Fenwick combination from sections 4 and 36. Keep original values separately from accumulated update differences.

6.2 Algorithm

```
record tin,tout with DFS
A u x: add +x at tin[u], and -x at tout[u] if within the array
Q u: original[u] + prefix(tin[u]) of the update difference array
```

6.3 Sample walkthrough

Adding 10 to subtree 3 changes vertices 3,4,5. Vertex 4 becomes 14, while vertex 2 stays 2. Subtracting one from the root's subtree then leaves vertex 3 at 12.

6.4 Correctness

Subtree contiguity makes each update an interval addition. Its two boundary events contribute x to a prefix exactly at positions inside that interval and zero outside it. Adding all preceding update events is linear, so the prefix at $\text{tin}[u]$ is the accumulated change to vertex u . Adding its initial value therefore yields its current value.

6.5 Complexity

Preprocessing $O(n)$; each operation $O(\log n)$; memory $O(n)$.

6.6 Implementation notes and targeted tests

A preorder subtree interval differs from an entry-and-exit tour of length $2n$. Handle the endpoint beyond the array explicitly. Test root updates, leaf updates, negative additions, and overlapping ancestor subtrees.

Use iterative DFS for the input tree in C++ and Python so a long chain does not exhaust the call stack. Segment-tree recursion has logarithmic depth.

6.7 C++ reference solution

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q))
        return 0;
    vector<ll> a(n);
    for (auto &x : a)
        cin >> x;
    vector<vector<int>> > g(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> tin(n), tout(n), par(n, -1), it(n);
    int timer = 0;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        if (!it[u])
            tin[u] = ++timer;
        if (it[u] < (int)g[u].size()) {
            int v = g[u][it[u]++];
            if (v == par[u])
                continue;
            par[v] = u;
            st.push_back(v);
        } else {
            tout[u] = timer;
            st.pop_back();
        }
    }
    vector<ll> bit(n + 2);
    auto add = [&](int i, ll x) {
        for (; i <= n + 1; i += i & -i)
            bit[i] += x;
    };
    auto get = [&](int i) {
        ll s = 0;
        for (; i; i -= i & -i)
            s += bit[i];
        return s;
    };
```

```

};
while (q--) {
    char c;
    int u;
    cin >> c >> u;
    --u;
    if (c == 'A') {
        ll x;
        cin >> x;
        add(tin[u], x);
        add(tout[u] + 1, -x);
    } else
        cout << a[u] + get(tin[u]) << '\n';
}
}

```

6.8 Python reference solution

```

import sys
def main():
    d=sys.stdin.buffer.read().split()
    if not d:return
    n,q=map(int,d[:2]);a=list(map(int,d[2:2+n]));g=[[]for _ in range(n)];p=2+n
    for _ in range(n-1):
        u,v=int(d[p])-1,int(d[p+1])-1;p+=2;g[u].append(v);g[v].append(u)
    tin=[0]*n;tout=[0]*n;parent=[-1]*n;timer=0;stack=[(0,0)]
    while stack:
        u,state=stack.pop()
        if state==0:
            timer+=1;tin[u]=timer;stack.append((u,1))
            for v in reversed(g[u]):
                if v!=parent[u]:parent[v]=u;stack.append((v,0))
        else:tout[u]=timer
    bit=[0]*(n+2)
    def add(i,x):
        while i<=n+1:bit[i]+=x;i+=i&-i
    def get(i):
        s=0
        while i:s+=bit[i];i-=i&-i
        return s
    out=[]
    for _ in range(q):
        c=d[p];u=int(d[p+1])-1;p+=2
        if c==b'A':x=int(d[p]);p+=1;add(tin[u],x);add(tout[u]+1,-x)
        else:out.append(str(a[u]+get(tin[u])))
    print('\n'.join(out))
if __name__=="__main__":main()

```

7 F. Dynamic Subtree Maximum

7.1 Contract and reduction

The query is a subtree maximum, while updates assign a single vertex. Flatten the rooted tree using section 41, then apply the ordinary point-update, range-query segment tree of section 37 to the Euler array. No tree-path decomposition is required.

7.2 Algorithm

```
flatten the tree; base[tin[u]] = initial[u]
build a maximum segment tree
U u x: assign base[tin[u]] = x
Q u: query maximum on [tin[u], tout[u])
```

7.3 Sample walkthrough

Subtree 3 has values 3, 4, 5, maximum 5. Assigning vertex 4 to -10 leaves maximum 5. Assigning vertex 5 to -2 leaves values 3, -10, -2, maximum 3.

7.4 Correctness

The Euler interval contains exactly the subtree vertices. Each segment-tree node stores the maximum of its interval. A point assignment changes one leaf; recomputing ancestors preserves the invariant. Query nodes partition the subtree interval, so taking their maximum gives the largest current subtree value. A leaf query includes the vertex itself.

7.5 Complexity

Build and flatten $O(n)$; updates and queries $O(\log n)$; memory $O(n)$.

7.6 Implementation notes and targeted tests

Zero is not a neutral maximum when values are negative. Assignment must replace, not add. Test an all-negative tree, decreasing the unique maximum, root and leaf queries, a single vertex, chains, and stars. DFS must be iterative for the largest chains; segment recursion remains logarithmic.

Use iterative DFS for the input tree in C++ and Python so a long chain does not exhaust the call stack. Segment-tree recursion has logarithmic depth.

7.7 C++ reference solution

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n,q; if (!(cin>>n>>q)) return 0;
    vector<ll> original(n); for (auto &x:original) cin>>x;
    vector<vector<int>> g(n);
    for (int i=1,u,v; i<n; ++i) {cin>>u>>v; --u; --v; g[u].push_back(v); g[v].push_back(u);}
    vector<int> tin(n),tout(n),order;
    vector<tuple<int,int,bool>> stack{{0,-1,false}};
    while (!stack.empty()) {
        auto [u,p,leaving]=stack.back(); stack.pop_back();
        if (leaving) {tout[u]=order.size(); continue;}
        tin[u]=order.size(); order.push_back(u); stack.emplace_back(u,p,true);
        for (auto it=g[u].rbegin(); it!=g[u].rend(); ++it)
            if (*it!=p) stack.emplace_back(*it,u,false);
    }
    int size=1;while(size<n)size*=2;
    vector<ll> tree(2*size,LLONG_MIN);
    for(int i=0;i<n;++i)tree[size+i]=original[order[i]];
    for(int z=size-1;z-->0)tree[z]=max(tree[2*z],tree[2*z+1]);
    auto assign=[&](int i,ll x){int z=size+i;tree[z]=x;for(z/=2;z>0;z/=2)tree[z]=max(tree[2*z],tree[2*z+1]);}
    auto query=[&](int l,int r){ll answer=LLONG_MIN;l+=size;r+=size;
        while(l<r){if(l&1)answer=max(answer,tree[l++]);if(r&1)answer=max(answer,tree[--r]);l++;r--;}
    }
    while(q--){char op;int u;cin>>op>>u;--u;
        if(op=='Q')cout<<query(tin[u],tout[u])<<'\n';
        else{ll x;cin>>x;assign(tin[u],x);}}
}
```

7.8 Python reference solution

```
import sys

def main():
```

```

data = sys.stdin.buffer.read().split()
if not data: return
n, q = int(data[0]), int(data[1]); pos = 2
original = list(map(int, data[pos:pos+n])); pos += n
g = [[] for _ in range(n)]
for _ in range(n - 1):
    u, v = int(data[pos]) - 1, int(data[pos + 1]) - 1; pos += 2
    g[u].append(v); g[v].append(u)
tin = [0] * n; tout = [0] * n; order = []; stack = [(0, -1, False)]
while stack:
    u, parent, leaving = stack.pop()
    if leaving:
        tout[u] = len(order); continue
    tin[u] = len(order); order.append(u)
    stack.append((u, parent, True))
    for v in reversed(g[u]):
        if v != parent: stack.append((v, u, False))
size = 1
while size < n: size *= 2
tree = [-10**30] * (2*size)
for i,u in enumerate(order): tree[size+i] = original[u]
for node in range(size-1,0,-1):
    tree[node] = max(tree[2*node],tree[2*node+1])
def assign(i,x):
    node=size+i;tree[node]=x;node//=2
    while node:
        tree[node]=max(tree[2*node],tree[2*node+1]);node//=2
def query(left,right):
    left+=size;right+=size;answer=-10**30
    while left<right:
        if left&1:answer=max(answer,tree[left]);left+=1
        if right&1:right-=1;answer=max(answer,tree[right])
        left//=2;right//=2
    return answer
out=[]
for _ in range(q):
    op=data[pos];u=int(data[pos+1])-1;pos+=2
    if op==b'Q':out.append(query(tin[u],tout[u]))
    else:x=int(data[pos]);pos+=1;assign(tin[u],x)
sys.stdout.write('\n'.join(map(str,out))+('\n' if out else ''))

if __name__ == "__main__": main()

```

8 G. Subtree Value Count

8.1 Contract and reduction

The values and all queries are static. Combine Euler intervals from section 41 with the offline threshold sweep from sections 13 and 36. Count values at most hi and subtract values at most $lo-1$ inside the same subtree interval.

8.2 Algorithm

```
flatten the tree and sort vertices by value
query (u,lo,hi) creates (hi,u,id,+1) and (lo-1,u,id,-1)
sort events by threshold
before event T, activate all vertices with value <= T
activation adds 1 at that vertex's Euler position in one Fenwick tree
answer[id] += sign * count in [tin[u],tout[u])
```

8.3 Sample walkthrough

Subtree 3 has values 4,2,3; the interval [2,3] selects two. The entire tree has two values in [4,5]. Vertex 2 alone has value 1, which is inside [0,10].

8.4 Correctness

At threshold T , the active positions are exactly the vertices with value at most T . The Fenwick interval count restricts that set to exactly the requested subtree. Subtracting the count at $lo-1$ removes all values below lo , while retaining values equal to either endpoint. Each vertex is activated once and contributes separately even if values repeat.

8.5 Complexity

DFS $O(n)$; sorting and sweep $O((n+q) \log(n+q))$; memory $O(n+q)$.

8.6 Implementation notes and targeted tests

Index the Fenwick by position, not value: values determine activation order, positions determine membership in the subtree. Activate all equal values before answering their threshold. Test duplicate values, $lo=hi$, no matches, all matches, and negative bounds. Queries must be restored to input order.

Use iterative DFS for the input tree in C++ and Python so a long chain does not exhaust the call stack. Segment-tree recursion has logarithmic depth.

8.7 C++ reference solution

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n,q; if (!(cin>>n>>q)) return 0;
    vector<ll> original(n); for (auto &x:original) cin>>x;
    vector<vector<int>> g(n);
    for (int i=1,u,v; i<n; ++i) {cin>>u>>v; --u; --v; g[u].push_back(v); g[v].push_back(u);}
    vector<int> tin(n),tout(n),order;
    vector<tuple<int,int,bool>> stack{{0,-1,false}};
    while (!stack.empty()) {
        auto [u,p,leaving]=stack.back(); stack.pop_back();
        if (leaving) {tout[u]=order.size(); continue;}
        tin[u]=order.size(); order.push_back(u); stack.emplace_back(u,p,true);
        for (auto it=g[u].rbegin(); it!=g[u].rend(); ++it)
            if (*it!=p) stack.emplace_back(*it,u,false);
    }
    vector<int> vertices(n);iota(vertices.begin(),vertices.end(),0);
    sort(vertices.begin(),vertices.end(),[&](int u,int v){return original[u]<original[v];});
    vector<tuple<ll,int,int,int>> events;vector<int> ans(q),bit(n+1);
    for(int i=0;i<q;++i){int u;ll lo,hi;cin>>u>>lo>>hi;--u;events.emplace_back(hi,u,i,1);eve
    sort(events.begin(),events.end());
    auto prefix=[&](int i){int total=0;for(;i;i-=i&-i)total+=bit[i];return total;};
    int current=0;
    for(auto [bound,u,id,sign]:events){
        while(current<n&&original[vertices[current]]<=bound){for(int j=tin[vertices[current]]
            ans[id]+=sign*(prefix(tout[u])-prefix(tin[u]));
        }
    }
    for(int x:ans)cout<<x<<'\\n';
}
```

8.8 Python reference solution

```

import sys

def main():
    data = sys.stdin.buffer.read().split()
    if not data: return
    n, q = int(data[0]), int(data[1]); pos = 2
    original = list(map(int, data[pos:pos+n])); pos += n
    g = [[] for _ in range(n)]
    for _ in range(n - 1):
        u, v = int(data[pos]) - 1, int(data[pos + 1]) - 1; pos += 2
        g[u].append(v); g[v].append(u)
    tin = [0] * n; tout = [0] * n; order = []; stack = [(0, -1, False)]
    while stack:
        u, parent, leaving = stack.pop()
        if leaving:
            tout[u] = len(order); continue
        tin[u] = len(order); order.append(u)
        stack.append((u, parent, True))
        for v in reversed(g[u]):
            if v != parent: stack.append((v, u, False))
    vertices = sorted(range(n), key=lambda u: original[u]); events = []; out = [0] * q
    for i in range(q):
        u, lo, hi = map(int, data[pos:pos+3]); pos += 3; u -= 1
        events.append((hi, u, i, 1)); events.append((lo-1, u, i, -1))
    bit = [0] * (n+1)
    def prefix(i):
        total = 0
        while i: total += bit[i]; i -= i & -i
        return total
    current = 0
    for bound, u, i, sign in sorted(events):
        while current < n and original[vertices[current]] <= bound:
            j = tin[vertices[current]] + 1
            while j <= n: bit[j] += 1; j += j & -j
            current += 1
        out[i] += sign * (prefix(tout[u]) - prefix(tin[u]))
    print('\n'.join(map(str, out)))

if __name__ == "__main__": main()

```

9 H. Incremental Connectivity

9.1 Contract and reduction

Only edges are added, so this is precisely the DSU contract from section 18. Maintain the connected components as disjoint sets. Use union by size and path compression; neither reverse time nor rollback is necessary.

9.2 Algorithm

```
start each vertex in its own component
A u v: find both representatives; union them if different
Q u v: print YES exactly when representatives are equal
```

9.3 Sample walkthrough

Initially 1 and 3 are separate. After edges 1-2 and 2-3, they share a component. Vertex 4 remains separate until edge 3-4 is added.

9.4 Correctness

Initially the components are isolated vertices, matching the empty graph. Adding an edge connects the two endpoint components and no others, exactly the effect of a DSU union. If they are already equal, a repeated edge or self-loop changes no component. Induction over insertions proves that the DSU sets are exactly the graph's connected components at every query.

9.5 Complexity

Initialization $O(n)$; all operations $O(q \alpha(n))$ amortized; memory $O(n)$. Here α is the inverse Ackermann function.

9.6 Implementation notes and targeted tests

Do not count a repeated edge as a new component merge. A vertex is connected to itself even without edges. Test repeated edges, self-loops, a growing chain, isolated vertices, and queries before the first insertion.

9.7 C++ reference solution

```
#include <bits/stdc++.h>
using namespace std;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,q;if(!(cin>>n>>q))return 0;
vector<int> p(n),sz(n,1);iota(p.begin(),p.end(),0);
auto find=[&](int u){while(p[u]!=u){p[u]=p[p[u]];u=p[u];}return u;};
while(q--){char op;int u,v;cin>>op>>u>>v;u=find(u-1);v=find(v-1);
if(op=='A'){if(u!=v){if(sz[u]<sz[v])swap(u,v);p[v]=u;sz[u]+=sz[v];}}
else cout<<(u==v?"YES":"NO")<<'\n';}}
```

9.8 Python reference solution

```
import sys

def main():
    data=sys.stdin.buffer.read().split()
    if not data:return
    n,q=map(int,data[:2]);parent=list(range(n));size=[1]*n;pos=2;out=[]
    def find(u):
        while parent[u]!=u:parent[u]=parent[parent[u]];u=parent[u]
        return u
    for _ in range(q):
        op=data[pos];u,v=int(data[pos+1])-1,int(data[pos+2])-1;pos+=3
        u,v=find(u),find(v)
        if op==b'A':
            if u!=v:
                if size[u]<size[v]:u,v=v,u
                parent[v]=u;size[u]+=size[v]
        else:out.append('YES' if u==v else 'NO')
    print('\n'.join(out))

if __name__ == "__main__": main()
```

10 I. Subtree Assign, Add, and Sum

10.1 Contract and reduction

Compose the Euler mapping from section 41 with the lazy sum tree used in B. The new task adds a tree-shaped query set; its assignment/addition semantics are unchanged. Convert each subtree to one interval, then reuse B's invariant.

10.2 Algorithm

```
flatten the tree and build the Euler array
1 u x: lazy-assign interval [tin[u],tout[u]) to x
2 u x: lazy-add x to that interval
3 u: query its sum
compose and push assignment/addition tags as in B
```

10.3 Sample walkthrough

Subtree 3 initially sums to 12. Assigning 7 to it gives 7,7,7; adding -2 to the leaf subtree of 4 leaves 7,5,7, sum 19. Vertices 1,2 add 3, so the entire tree sums to 22.

10.4 Correctness

Euler contiguity gives exactly the affected vertex set for each operation. B's tag rules assign or add the specified amount to every entry in that interval, maintaining every node sum. Since the Euler mapping is one-to-one, the interval sum equals the sum of the subtree's current vertex values. The mapping and the lazy-tree proof establish separate parts of correctness.

10.5 Complexity

Flatten and build $O(n)$; each operation $O(\log n)$; memory $O(n)$.

10.6 Implementation notes and targeted tests

Keep interval conventions consistent: these implementations adapt B's inclusive 1-based segment-tree interval to $[tin[u]+1, tout[u]]$. Test assignments to zero, an ancestor assignment after a descendant addition, the opposite order, leaves, and whole-tree operations. Use 64-bit products.

Use iterative DFS for the input tree in C++ and Python so a long chain does not exhaust the call stack. Segment-tree recursion has logarithmic depth.

10.7 C++ reference solution

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int n, q;
vector<ll> sm, addv, asgv;
vector<char> hasa;

static void put_assign(int x, int len, ll v) {
    sm[x] = v * len; asgv[x] = v; hasa[x] = 1; addv[x] = 0;
}
static void put_add(int x, int len, ll v) {
    sm[x] += v * len;
    if (hasa[x]) asgv[x] += v; else addv[x] += v;
}
static void push(int x, int l, int r) {
    int m = (l + r) / 2, lc = 2 * x, rc = 2 * x + 1;
    if (hasa[x]) {
        put_assign(lc, m - l + 1, asgv[x]);
        put_assign(rc, r - m, asgv[x]);
        hasa[x] = 0; asgv[x] = 0;
    }
    if (addv[x]) {
        put_add(lc, m - l + 1, addv[x]);
        put_add(rc, r - m, addv[x]);
        addv[x] = 0;
    }
}
static void build(int x, int l, int r, vector<ll> &a) {
    if (l == r) { sm[x] = a[l]; return; }
    int m = (l + r) / 2;
    build(2 * x, l, m, a); build(2 * x + 1, m + 1, r, a);
    sm[x] = sm[2 * x] + sm[2 * x + 1];
}
static void upd(int x, int l, int r, int ql, int qr,
               ll v, bool assign_) {
    if (qr < l || r < ql) return;
```

```

    if (ql <= l && r <= qr) {
        if (assign_) put_assign(x, r - l + 1, v);
        else put_add(x, r - l + 1, v);
        return;
    }
    push(x, l, r);
    int m = (l + r) / 2;
    upd(2 * x, l, m, ql, qr, v, assign_);
    upd(2 * x + 1, m + 1, r, ql, qr, v, assign_);
    sm[x] = sm[2 * x] + sm[2 * x + 1];
}

static ll qry(int x, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return sm[x];
    push(x, l, r);
    int m = (l + r) / 2;
    return qry(2 * x, l, m, ql, qr) + qry(2 * x + 1, m + 1, r, ql, qr);
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    if (!(cin >> n >> q)) return 0;
    vector<ll> original(n); for(auto &x:original) cin>>x;
    vector<vector<int>> g(n);
    for (int i=1,u,v; i<n; ++i) {cin>>u>>v; --u; --v; g[u].push_back(v); g[v].push_back(u);}
    vector<int> tin(n),tout(n),order;
    vector<tuple<int,int,bool>> stack{{0,-1,false}};
    while (!stack.empty()) {
        auto [u,p,leaving]=stack.back(); stack.pop_back();
        if (leaving) {tout[u]=order.size(); continue;}
        tin[u]=order.size(); order.push_back(u); stack.emplace_back(u,p,true);
        for (auto it=g[u].rbegin(); it!=g[u].rend(); ++it)
            if (*it!=p) stack.emplace_back(*it,u,false);
    }
    vector<ll> a(n+1); for(int i=0;i<n;++i) a[i+1]=original[order[i]];
    sm.assign(4 * n + 4, 0); adv.assign(4 * n + 4, 0);
    asgv.assign(4 * n + 4, 0); hasa.assign(4 * n + 4, 0);
    build(1, 1, n, a);
    string out;
    while (q--) {
        int t,u; ll x; cin>>t>>u; --u;
        int l=tin[u]+1,r=tout[u];
        if(t==3) out+=to_string(qry(1,1,n,l,r))+"\n";
        else {cin>>x;upd(1,1,n,l,r,x,t==1);}
    }
    cout << out;
}

```

10.8 Python reference solution

```
import sys

def main():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    pos = 0
    n = int(data[pos]); q = int(data[pos + 1]); pos += 2
    original = list(map(int, data[pos:pos+n])); pos += n
    g = [[] for _ in range(n)]
    for _ in range(n - 1):
        u, v = int(data[pos]) - 1, int(data[pos + 1]) - 1; pos += 2
        g[u].append(v); g[v].append(u)
    tin = [0] * n; tout = [0] * n; order = []; stack = [(0, -1, False)]
    while stack:
        u, parent, leaving = stack.pop()
        if leaving:
            tout[u] = len(order); continue
        tin[u] = len(order); order.append(u)
        stack.append((u, parent, True))
        for v in reversed(g[u]):
            if v != parent: stack.append((v, u, False))
    a = [0] + [original[u] for u in order]

    size = 4 * n + 4
    sm = [0] * size
    addv = [0] * size
    asgv = [0] * size
    hasa = [False] * size

    def put_assign(x, ln, v):
        sm[x] = v * ln; asgv[x] = v; hasa[x] = True; addv[x] = 0

    def put_add(x, ln, v):
        sm[x] += v * ln
        if hasa[x]:
            asgv[x] += v
        else:
            addv[x] += v

    def push(x, l, r):
        m = (l + r) // 2; lc = 2 * x; rc = 2 * x + 1
        if hasa[x]:
            put_assign(lc, m - l + 1, asgv[x])
            put_assign(rc, r - m, asgv[x])
            hasa[x] = False; asgv[x] = 0
        if addv[x]:
            put_add(lc, m - l + 1, addv[x])
```

```

        put_add(rc, r - m, adv[x])
        adv[x] = 0

def build(x, l, r):
    if l == r:
        sm[x] = a[l]; return
    m = (l + r) // 2
    build(2 * x, l, m); build(2 * x + 1, m + 1, r)
    sm[x] = sm[2 * x] + sm[2 * x + 1]

def upd(x, l, r, ql, qr, v, is_assign):
    if qr < l or r < ql:
        return
    if ql <= l and r <= qr:
        if is_assign:
            put_assign(x, r - l + 1, v)
        else:
            put_add(x, r - l + 1, v)
        return
    push(x, l, r)
    m = (l + r) // 2
    upd(2 * x, l, m, ql, qr, v, is_assign)
    upd(2 * x + 1, m + 1, r, ql, qr, v, is_assign)
    sm[x] = sm[2 * x] + sm[2 * x + 1]

def qry(x, l, r, ql, qr):
    if qr < l or r < ql:
        return 0
    if ql <= l and r <= qr:
        return sm[x]
    push(x, l, r)
    m = (l + r) // 2
    return (qry(2 * x, l, m, ql, qr)
            + qry(2 * x + 1, m + 1, r, ql, qr))

sys.setrecursionlimit(300000)
build(1, 1, n)
out = []
for _ in range(q):
    t = int(data[pos]); pos += 1
    u = int(data[pos]) - 1; pos += 1
    l, r = tin[u]+1, tout[u]
    if t == 3: out.append(qry(1,1,n,l,r))
    else:
        x = int(data[pos]); pos += 1
        upd(1,1,n,l,r,x,t == 1)
sys.stdout.write("\n".join(map(str, out)) + ("\n" if out else ""))

if __name__ == "__main__":
    main()

```

11 J. Static Vertex Path Sums

11.1 Contract and reduction

The vertex values never change, so root-prefix sums suffice alongside LCA from section 41. Let $S[u]$ include every vertex from root 1 through u . For $w = \text{lca}(u, v)$, the required sum is $S[u] + S[v] - 2 * S[w] + a[w]$.

11.2 Algorithm

```
DFS: compute parent, depth, and  $S[\text{child}] = S[\text{parent}] + a[\text{child}]$ 
build the usual binary-lifting ancestor table
for query  $u, v$ :
     $w = \text{lca}(u, v)$ 
    print  $S[u] + S[v] - 2 * S[w] + a[w]$ 
```

11.3 Sample walkthrough

For query 2,4, the LCA is 1: root sums are 3 and 8, so $3+8-2*1+1=10$. For 4,5, the LCA is 3; the answer is $4+3+5=12$. For 3,3, the formula returns $a[3]=3$.

11.4 Correctness

The two root prefixes both contain the root-to- w prefix. Subtracting it twice leaves the two branches strictly below w ; neither branch contains w . Adding $a[w]$ restores that shared endpoint once. The two branches and w partition the simple path, proving the formula, including $u=v$.

11.5 Complexity

Preprocessing $O(n \log n)$; each query $O(\log n)$; memory $O(n \log n)$.

11.6 Implementation notes and targeted tests

Do not apply an edge-distance formula directly to inclusive vertex values: the shared ancestor must be restored. This formula relies on static values; assignments would invalidate precomputed root sums. Test equal endpoints, ancestor endpoints, negative sums, an LCA at the root, and a long chain.

Use iterative DFS for the input tree in C++ and Python so a long chain does not exhaust the call stack. Segment-tree recursion has logarithmic depth.

11.7 C++ reference solution

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n,q; if (!(cin>>n>>q)) return 0;
    vector<ll> original(n); for (auto &x:original) cin>>x;
    vector<vector<int>> g(n);
    for (int i=1,u,v; i<n; ++i) {cin>>u>>v; --u; --v; g[u].push_back(v); g[v].push_back(u);}
    vector<int> tin(n),tout(n),order;
    vector<tuple<int,int,bool>> stack{{0,-1,false}};
    while (!stack.empty()) {
        auto [u,p,leaving]=stack.back(); stack.pop_back();
        if (leaving) {tout[u]=order.size(); continue;}
        tin[u]=order.size(); order.push_back(u); stack.emplace_back(u,p,true);
        for (auto it=g[u].rbegin(); it!=g[u].rend(); ++it)
            if (*it!=p) stack.emplace_back(*it,u,false);
    }
    vector<int> parent(n),depth(n);vector<ll> sums(n);sums[0]=original[0];
    for(int u:order)for(int v:g[u])if(v!=parent[u]){parent[v]=u;depth[v]=depth[u]+1;sums[v]=sums[u]+original[v];}
    int levels=1;while((1<=levels)&&n>=levels)++levels;
    vector<vector<int>> up(levels,parent);
    for(int k=1;k<levels;++k)for(int u=0;u<n;++u)up[k][u]=up[k-1][up[k-1][u]];
    auto lca=[&](int u,int v){
        if(depth[u]<depth[v])swap(u,v);int diff=depth[u]-depth[v];
        for(int k=0;k<levels;++k)if(diff>>k&1)u=up[k][u];
        if(u==v)return u;
        for(int k=levels-1;k>=0;--k)if(up[k][u]!=up[k][v]){u=up[k][u];v=up[k][v];}
        return parent[u];
    };
    while(q--){int u,v;cin>>u>>v;--u;--v;int w=lca(u,v);cout<<sums[u]+sums[v]-2*sums[w]+original[w]<<endl;}
}
```

11.8 Python reference solution

```
import sys

def main():
    data = sys.stdin.buffer.read().split()
    if not data: return
    n, q = int(data[0]), int(data[1]); pos = 2
    original = list(map(int, data[pos:pos+n])); pos += n
    g = [[] for _ in range(n)]
    for _ in range(n - 1):
        u, v = int(data[pos]) - 1, int(data[pos + 1]) - 1; pos += 2
        g[u].append(v); g[v].append(u)
    tin = [0] * n; tout = [0] * n; order = []; stack = [(0, -1, False)]
    while stack:
        u, parent, leaving = stack.pop()
        if leaving:
            tout[u] = len(order); continue
        tin[u] = len(order); order.append(u)
        stack.append((u, parent, True))
        for v in reversed(g[u]):
            if v != parent: stack.append((v, u, False))
    parent=[0]*n;depth=[0]*n;sums=[0]*n;sums[0]=original[0]
    for u in order:
        for v in g[u]:
            if v!=parent[u]:parent[v]=u;depth[v]=depth[u]+1;sums[v]=sums[u]+original[v]
    up=[parent]
    for _ in range(1,n.bit_length()):
        previous=up[-1];up.append([previous[previous[u]] for u in range(n)])
    def lca(u,v):
        if depth[u]<depth[v]:u,v=v,u
        diff=depth[u]-depth[v]
        for k in range(len(up)):
            if diff>>k&1:u=up[k][u]
        if u==v:return u
        for k in range(len(up)-1,-1,-1):
            if up[k][u]!=up[k][v]:u,v=up[k][u],up[k][v]
        return parent[u]
    out=[]
    for _ in range(q):
        u,v=int(data[pos])-1,int(data[pos+1])-1;pos+=2;w=lca(u,v)
        out.append(sums[u]+sums[v]-2*sums[w]+original[w])
    print('\n'.join(map(str,out)))

if __name__ == "__main__": main()
```

12 K. Static Weighted Rectangles

12.1 Contract and reduction

Reuse C's offline rectangle reduction, but insert the weight of each point rather than the count 1. The point set is fixed; this task needs only one ordinary Fenwick tree. Negative weights do not affect the sweep order.

12.2 Algorithm

```
sort fixed points by x; compress their y coordinates
create +1 event at c and -1 event at a-1 for each rectangle [a,c] x [b,d]
insert each eligible point's weight at its y rank
for each event, accumulate its sign times the Fenwick sum in [b,d]
restore query order
```

12.3 Sample walkthrough

Weights 5 and -2 at (1,1) contribute 3 together. Including the point of weight 4 at (2,3) gives the first answer 7. The second rectangle includes only that point, giving 4; the third gives 3.

12.4 Correctness

The active Fenwick prefix is the sum of weights with bounded x and y, because each eligible point is inserted exactly once. Prefix subtraction selects the inclusive y-range. Subtracting the event at $a-1$ removes the contribution from points left of the rectangle. This is a linear identity for weights of either sign, and coincident points remain separate contributions.

12.5 Complexity

Sorting and processing $O((n+q) \log(n+q))$; memory $O(n+q)$.

12.6 Implementation notes and targeted tests

Use 64-bit Fenwick sums and answers. Do not discard negative weights or combine coincident points by keeping just one weight. Query bounds can be absent from compression. Test cancellations, all-negative weights, coincident points, degenerate rectangles, and coordinates near both extremes.

12.7 C++ reference solution

```
#include <bits/stdc++.h>
using namespace std; using ll = long long;
struct BIT { vector<ll> b; BIT(int n) : b(n+1) {} void add(int i, ll x) { for(; i < (int)b.size(); i += i & - i) b[i] += x; }
int main() { ios::sync_with_stdio(false); cin.tie(nullptr); int n, q; if(!(cin >> n >> q)) return 0;
vector<tuple<ll, ll, ll>> points; vector<ll> ys;
for(int i = 0; i < n; ++i) { ll x, y, w; cin >> x >> y >> w; points.emplace_back(x, y, w); ys.push_back(y); }
vector<tuple<ll, ll, ll, int, int>> events; vector<ll> ans(q);
for(int i = 0; i < q; ++i) { ll a, b, c, d; cin >> a >> b >> c >> d; events.emplace_back(c, b, d, i, 1); events.emplace_back(c, b, d, i, -1);
sort(points.begin(), points.end()); sort(events.begin(), events.end()); sort(ys.begin(), ys.end());
for(auto [x, lo, hi, id, sign] : events) { while(p < n && get<0>(points[p]) <= x) { auto [px, y, w] = points[p++];
int l = lower_bound(ys.begin(), ys.end(), lo) - ys.begin(), r = upper_bound(ys.begin(), ys.end(), hi) - ys.begin();
for(ll x : ans) cout << x << '\n'; }
```

12.8 Python reference solution

```
import sys, bisect
class BIT:
    def __init__(self, n): self.n = n; self.bit = [0] * (n+1)
    def add(self, i, x):
        while i <= self.n: self.bit[i] += x; i += i & - i
    def sum(self, i):
        s = 0
        while i > 0: s += self.bit[i]; i -= i & - i
        return s
def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data: return
    n, q = data[0], data[1]; idx = 2; pts = []; ys = []
    for _ in range(n): x, y, w = data[idx:idx+3]; idx += 3; pts.append((x, y, w)); ys.append(y)
    events = []; ans = [0] * q
    for qi in range(q):
        x1, y1, x2, y2 = data[idx], data[idx+1], data[idx+2], data[idx+3]; idx += 4; ys += [y1, y2]
        events.append((x2, y1, y2, qi, 1)); events.append((x1-1, y1, y2, qi, -1))
    ys = sorted(set(ys)); pts.sort(); events.sort(); bit = BIT(len(ys)); p = 0
    for x, y1, y2, qi, sgn in events:
        while p < n and pts[p][0] <= x:
            bit.add(bisect.bisect_left(ys, pts[p][1]) + 1, pts[p][2]); p += 1
            l = bisect.bisect_left(ys, y1) + 1; r = bisect.bisect_right(ys, y2)
            ans[qi] += sgn * (bit.sum(r) - bit.sum(l-1))
        print("\n".join(map(str, ans)))
if __name__ == "__main__": main()
```

13 L. Subtree Add, Subtree Maximum

13.1 Contract and reduction

Map subtrees to Euler intervals and use section 37's lazy range-add tree with a maximum summary. Uniform addition preserves which value is largest: $\max(a_i+x)=\max(a_i)+x$. Thus adding to a fully covered node changes its maximum by x , rather than by $x*\text{length}$ as a sum tree would.

13.2 Algorithm

```
flatten the tree; build maximum summaries
apply(node,x): maximum += x; pending_add += x
partial operation: push pending_add to both children; clear it; recurse
A u x: range-add on [tin[u],tout[u])
Q u: range-maximum on that interval
```

13.3 Sample walkthrough

Subtree 3 starts with maximum 5. Adding 5 gives values 8,9,10, so the whole-tree maximum is 10. Subtracting 20 from the whole tree leaves subtree 3 at -12,-11,-10, maximum -10.

13.4 Correctness

Each node maximum includes all updates covering its full interval. Applying a uniform addition shifts that maximum by exactly the added amount. Pushing shifts both child summaries, preserving their actual values; merging with maximum restores the parent summary after partial updates. Euler contiguity selects exactly the subtree vertices, so the range operation has the required effect and returns exactly the subtree maximum.

13.5 Complexity

Flatten and build $O(n)$; each operation $O(\log n)$; memory $O(n)$.

13.6 Implementation notes and targeted tests

Adding `x*length` to a maximum confuses it with a sum. Push before a partial query as well as before a partial update. Zero is not a neutral maximum. Test all-negative answers, root additions followed by leaf queries, nested subtree updates, cancellation, chains, and stars.

Use iterative DFS for the input tree in C++ and Python so a long chain does not exhaust the call stack. Segment-tree recursion has logarithmic depth.

13.7 C++ reference solution

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n,q; if (!(cin>>n>>q)) return 0;
    vector<ll> original(n); for (auto &x:original) cin>>x;
    vector<vector<int>> g(n);
    for (int i=1,u,v; i<n; ++i) {cin>>u>>v; --u; --v; g[u].push_back(v); g[v].push_back(u);}
    vector<int> tin(n),tout(n),order;
    vector<tuple<int,int,bool>> stack{{0,-1,false}};
    while (!stack.empty()) {
        auto [u,p,leaving]=stack.back(); stack.pop_back();
        if (leaving) {tout[u]=order.size(); continue;}
        tin[u]=order.size(); order.push_back(u); stack.emplace_back(u,p,true);
        for (auto it=g[u].rbegin(); it!=g[u].rend(); ++it)
            if (*it!=p) stack.emplace_back(*it,u,false);
    }
    vector<ll> mx(4*n),lazy(4*n);
    function<void(int,int,int)> build=[&](int z,int l,int r) {
        if(r-l==1){mx[z]=original[order[l]];return;}
        int m=(l+r)/2;build(z*2,l,m);build(z*2+1,m,r);mx[z]=max(mx[z*2],mx[z*2+1]);
    };
    auto apply=[&](int z,ll x){mx[z]+=x;lazy[z]+=x;};
    auto push=[&](int z){if(lazy[z]){apply(z*2,lazy[z]);apply(z*2+1,lazy[z]);lazy[z]=0;}};
    function<void(int,int,int,int,int,int,ll,bool)> update=[&](int z,int l,int r,int a,int b,ll
        if(b<=l||r<=a)return;
        if(a<=l&&r<=b){if(assign){mx[z]=x;lazy[z]=0;}else apply(z,x);return;}
        push(z);int m=(l+r)/2;update(z*2,l,m,a,b,x,assign);update(z*2+1,m,r,a,b,x,assign);mx
    };
    function<ll(int,int,int,int,int)> query=[&](int z,int l,int r,int a,int b)->ll {
        if(b<=l||r<=a)return LLONG_MIN;
        if(a<=l&&r<=b)return mx[z];
        push(z);int m=(l+r)/2;return max(query(z*2,l,m,a,b),query(z*2+1,m,r,a,b));
    };
    build(1,0,n);
    while(q--){char op;int u;cin>>op>>u;--u;if(op=='Q')cout<<query(1,0,n,tin[u],tout[u])<<'\\
```

```

    else{ll x;cin>>x;update(1,0,n,tin[u],tout[u],x,false);}}
}

```

13.8 Python reference solution

```

import sys

def main():
    data = sys.stdin.buffer.read().split()
    if not data: return
    n, q = int(data[0]), int(data[1]); pos = 2
    original = list(map(int, data[pos:pos+n])); pos += n
    g = [[] for _ in range(n)]
    for _ in range(n - 1):
        u, v = int(data[pos]) - 1, int(data[pos + 1]) - 1; pos += 2
        g[u].append(v); g[v].append(u)
    tin = [0] * n; tout = [0] * n; order = []; stack = [(0, -1, False)]
    while stack:
        u, parent, leaving = stack.pop()
        if leaving:
            tout[u] = len(order); continue
        tin[u] = len(order); order.append(u)
        stack.append((u, parent, True))
        for v in reversed(g[u]):
            if v != parent: stack.append((v, u, False))
    values = [original[u] for u in order]
    mx = [0] * (4*n); lazy = [0] * (4*n)
    def build(node, left, right):
        if right-left == 1: mx[node] = values[left]; return
        mid = (left+right)//2
        build(2*node,left,mid); build(2*node+1,mid,right)
        mx[node] = max(mx[2*node],mx[2*node+1])
    def apply(node, delta):
        mx[node] += delta; lazy[node] += delta
    def push(node):
        if lazy[node]:
            apply(2*node,lazy[node]); apply(2*node+1,lazy[node]); lazy[node] = 0
    def update(node,left,right,ql,q,r,x,assignment=False):
        if q>r or right<=ql: return
        if ql<=left and right<=qr:
            if assignment: mx[node]=x; lazy[node]=0
            else: apply(node,x)
            return
        push(node); mid=(left+right)//2
        update(2*node,left,mid,ql,q,r,x,assignment)
        update(2*node+1,mid,right,ql,q,r,x,assignment)
        mx[node]=max(mx[2*node],mx[2*node+1])
    def query(node,left,right,ql,q,r):

```

```

        if qr<=left or right<=ql: return -10**30
        if ql<=left and right<=qr: return mx[node]
        push(node); mid=(left+right)//2
        return max(query(2*node,left,mid,ql,qr),query(2*node+1,mid,right,ql,qr))
    build(1,0,n); out=[]
    for _ in range(q):
        op=data[pos]; u=int(data[pos+1])-1; pos+=2
        if op == b'Q': out.append(query(1,0,n,tin[u],tout[u]))
        else:
            x=int(data[pos]); pos+=1
            update(1,0,n,tin[u],tout[u],x)
    sys.stdout.write('\n'.join(map(str,out)) + ('\n' if out else ''))

if __name__ == "__main__": main()

```

14 Stress testing and what to carry forward

The package generators compare against direct multiset/array scans, explicit subtree enumeration, path reconstruction, rectangle scans, and graph searches. These oracles avoid reproducing the optimized decomposition. Fixed samples check the stated examples; random tests emphasize boundaries and update order. Use large chains and large operation counts to check stack safety and cost.

Run both reference languages with:

```
CP_TARGET=solution python3 sections/42_advanced_data_structures_mixed/check.py --keep-going
```

Carry forward the habit of proving the mapping separately from the stored invariant. The same established structure answers different questions after a valid reduction, but a convenient-looking interval or prefix formula is not sufficient without that proof.